Ruby offers two ways to access random elements of an array: the sample method and the shuffle method. The sample method returns a random member of the array, with an optional argument of how many elements to return. No matter how big the optional argument is, sample will not return the same element twice, so the effective limit on the argument is the size of the array. The duplicate restriction is by index, not value, so if the array contains duplicate values, so will the sample and in the same proportion.

The shuffle method returns a new array with the elements in a random order; x.shuffle is effectively equivalent to x.sample(x.size). Both methods take an optional keyword argument random:, which is expected to contain a Random instance that is used as the generator for the random ordering.

Unlike Enumerable but like Strings, Array implements ! versions of many of its methods where the original method returns a new array, and the ! version has the same logic but modifies the original array in place. In most of these cases, the return value is the original array if it has been modified, or nil if it has not.

In general, the reason to use the ! version is for performance reasons. They don't create a new array and therefore, they might be faster or use less memory. They tend to be more confusing, though, so we recommend you use only when needed.

The following ! methods related to methods we've discussed in this section or in Enumerable, on page 535 are all defined:

- compact!
- filter! (and also select!)
- flatten!
- map! (and also collect!)
- reject!
- reverse!
- rotate!
- shuffle!
- slice!
- sort!
- sort_by!
- uniq!

Array Math

In addition to the count method, arrays define length and size, which are aliases of each other, take no argument, and just return the raw element count. The empty? method returns true if the size of the array is zero.

Arrays are Comparable and define the <=> operator. Array comparisons in Ruby are managed element by element. So if a[0] <=> b[0] is not 0 return that, then a[1] <=> b[1] and so on until one array ends or there is a non-equal pair. The first non-equal pair is the result of the comparison. If all the pairs are equal, the shorter array is "less than" the larger array. So, equality for two arrays means that the arrays are the same size and all their elements are equal. For example:

```
[1, 2, 3] <=> [2, 2, 3]    # => -1
[1, 2, 3] <=> [1, 1, 3]    # => 1
[1, 2, 3] <=> [1, 2, 3, 4] # => -1
```